DETECTOR DE VAZAMENTO DE GÁS

com ESP32, Sensor MQ2 e Plataforma Blynk

Guia Passo a Passo de Montagem e Programação

Material Didático — Projeto de Internet das Coisas (IoT)

Julho de 2026

Por: Aquiles Angelo Rocha de Souza
Ciências da Computação

Guia de montagem e configuração passo a passo.

Projeto de detecção de vazamento de gás com notificação em tempo real no celular, usando ESP32, sensor MQ-2 e a plataforma Blynk IoT.

1. Materiais necessários

Componente	*Quantidade*	*Observação*
Placa ESP 32 (Dev Board)	1	Com suporte a Wi-Fi
Sensor de gás MQ-2	1	Módulo com saída analógica (AOUT) e digital (DOUT)
Protoboard	1	Para montagem sem solda
Jumpers macho-macho e macho-femea	5	Para as conexões VCC, GND e sinal
Cabo Micro USB de dados (não pode ser cabo só de carga)	1	Para alimentação e upload do código
Computador com Arduino IDE instalada	1	Versão mais recente instalada
Conta gratuita no Blynk (app no celular + console web)	1	Gratuita, criada em blynk.io

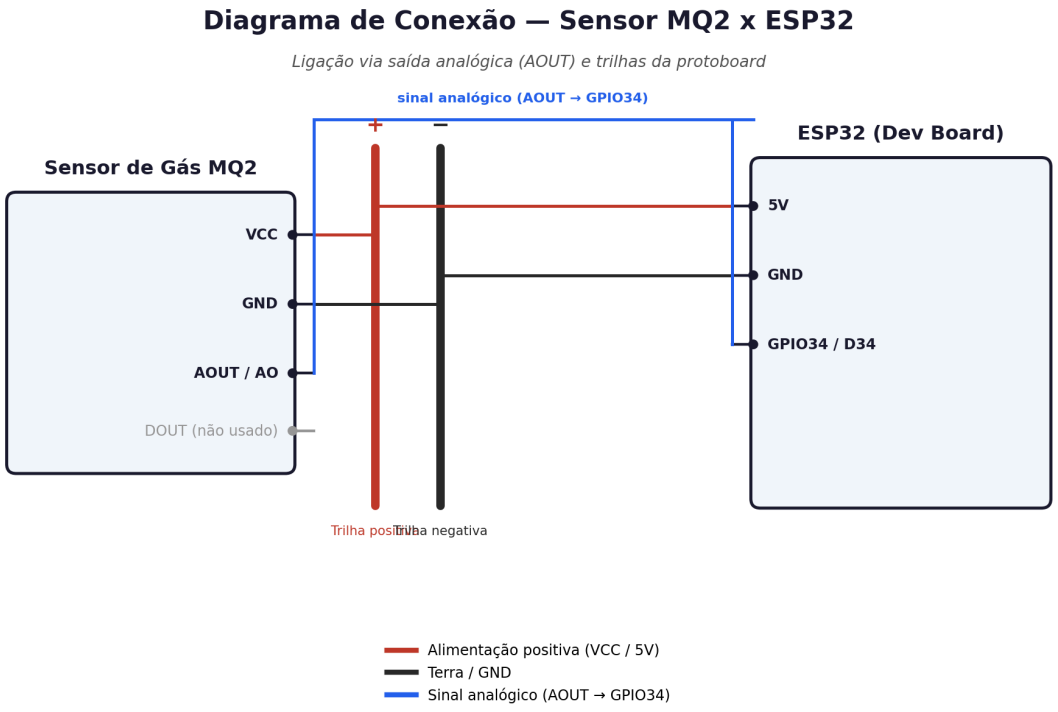
2. Montagem do circuito

O sensor MQ2 possui 4 pinos: VCC, GND, DOUT (saída digital) e AOUT (saída analógica). Nesta montagem usamos apenas a saída digital (DOUT), que informa apenas se o gás ultrapassou o limiar de sensibilidade ajustado no próprio módulo.

Observe as conexões:

Pinos MQ-2	Onde conectar
VCC (Pino de energia positiva)	Trilha positiva (+)
GND (Pino de energia negativa - terra)	Trilha negativa (-)
Pino AOUT ou AO (Analógico)	Pino GPIO34 ou D34 do ESP32

Pinos ESP32	Onde conectar
GND (Pino de energia negativa - terra)	Trilha negativa (-)
5V	Trilha positiva (+)
Pino GPIO34 ou D34 do ESP32	Pino AOUT ou AO (Analógico) do MQ2



Observação: o módulo tem um pequeno parafuso azul (trimpot) que ajusta a

sensibilidade da saída digital. Ele será calibrado na seção 7 deste guia.

3. Configuração do Arduino IDE + Drivers

Após a instalação do programa Arduino IDE, segue o passo a passo.

Passo a passo:

1. → Para adicionar o pacote de placas ESP32 ao arduino IDE → **File** no canto superior esquerdo → **Preferences** e cole a seguinte URL no campo "Additional boards manager URLs" e aperte ok.

(https://espressif.github.io/arduino-esp32/package_esp32_index.json)

2. → **Tools** na guia superior esquerda → **Board** → **Board manager** → digite "esp32" e baixe a versão (esp32 by Espressif Systems)

3. → Faça download do **DRIVER**: (https://www.youtube.com/redirect?event=video_description&redir_token=QUM4Zm9rVGM4eWlLNVZvaEpxclJfTTFBSXphb3xBR3JiS2FuSzVTTmFtN1RTNC13OXVaMFdFc19OSE43RmpMM3BubVJxWXpaQ1BabUxmbVhOelFqaVBScjVYbWdfOG43VjFvcjI0d2tuU01kMmdpNU5XR1hFWEpWMjc3NzN5SnFv&q=https%3A%2F%2Fwww.silabs.com%2Fdocuments%2Fpublic%2Fsoftware%2FCP210x_VCP_Windows.zip&v=LWXl4gQffs0) ps: clique para instalar o arquivo com x64.exe escrito no final.

4. → **Tools** na guia superior esquerda → **Board** → **esp32** → selecione a versão, muito provável que seja Node32s.

5. → **Tools** na guia superior esquerda → **Port** → escolha a porta (ex: COM3, COM1, etc) na qual o cabo Micro USB está conectado (entrada usb do computador), caso não saiba, acesse o gerenciador de dispositivos clicando no botão direito do mouse em cima do botão iniciar para conferir.

→ ***Conecte a placa ESP32 no seu computador!***

Ótimo! Temos quase tudo configurado, precisamos agora realizar a conexão com o Blynk.

6. → Instale a biblioteca Blynk na IDE → **Sketch** → **Include library** → **Manage**

libraries → pesquise por blynk e instale a biblioteca oficial do autor Volodymyr Shymanskyy.

6.1 → Dentro de `Documentos\Arduino\libraries\Blynk\src`, tem que existir o arquivo `BlynkSimpleEsp32.h`. Se não existir, a instalação ficou corrompida — desinstale e instale de novo pelo Gerenciador.

Tutorial para resolução do problema, caso não seja seu caso pule para o passo número 7.

6.2 → (**Caso tenha corrompido**) Descubra onde a IDE está salvando as bibliotecas Na Arduino IDE, vá em **Arquivo** → **Preferências** e olha o campo **"Localização do sketchbook"**. É dentro dessa pasta que fica a subpasta `libraries`.

Geralmente é: C:\Users\nome_do_usuário\Documents\Arduino\libraries

6.3 → A pasta `libraries` existe dentro de `Arduino`? Se existir, tem alguma pasta com "Blynk" no nome dentro dela? Se tiver, o que tem dentro dessa pasta Blynk (tem uma subpasta `src`? o que tem dentro do `src`?) Dentro dessa pasta `src`, tem um arquivo chamado `BlynkSimpleEsp32.h` especificamente?

Se não está, apagamos essa pasta blynk e reinstalamos do zero.

6.4 → **Apague a pasta Blynk inteira.**

Em C:\Users\nome_do_usuário\Documents\Arduino\libraries, apague a pasta `blynk` completa.

Baixe a biblioteca direto do GitHub

Acesse: https://github.com/blynkkk/blynk-library/releases/latest
Baixe o arquivo `.zip` do código-fonte (Source code (zip))

Instale pela IDE
Na Arduino IDE: **Sketch** → **Incluir Biblioteca** → **Adicionar Biblioteca .ZIP…**
Selecione o arquivo que você baixou.

7. → Coloque o código na IDE

Ótimo! O próximo passo é configurar o Blynk e conectá-lo diretamente ao código.

4. Configuração da plataforma Blynk IoT

8. → Acesse o Blynk pelo site no seu computador e crie uma conta

9. → No menu à esquerda vá em **zona do desenvolvedor** → **+ novo modelo** (botão verde no canto superior direito) → coloque um nome → no campo de hardware, selecione **ESP32** → no campo de tipo de conexão, selecione **WIFI.**

10. → No menu à esquerda vá em **dispositivos** → **+ novo dispositivo** (botão verde no canto superior direito) → **a partir do modelo** → no campo de escolha de modelo, selecione o modelo criado anteriormente.

11. → Após a criação do dispositivo ele vai indicar no canto superior direito o **ID do modelo, o nome do modelo e o AuthToken** que devem ser declarados na parte **superior do código** do firmware. Copie essas três informações indicadas e cole no início do seu código para estabelecer a conexão entre o **ESP32** e o aplicativo.

12. → Volte em **zona do desenvolvedor** e selecione o o modelo que criou anteriormente → no painel ao lado esquerdo vá em **eventos e notificações** (ícone de sino) → **+ adicionar novo evento** → **custom** → nomeie como “gas_alert”, tanto o nome do evento quanto o código do evento.

13. → Em tipo de dispositivo, selecione “Aviso” ou “Crítico”.

14. → Na aba de notificações, ative as notificações → no campo de **notificações por push** selecione proprietário do dispositivo.

15. → Na aba de configurações, em “***o evento será enviado ao usuário apenas uma vez por***” selecione

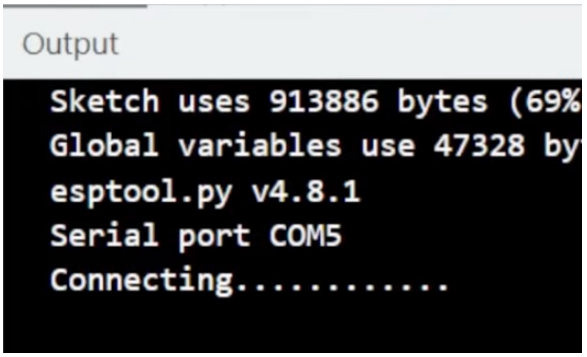
1 segundo. Em "***Mostrar na seção de Notificações e Eventos***" ative a página **do dispositivo** e **tela inicial.**

Ótimo! Volte no Arduino IDE para rodar o código e calibrar o MQ-2.

5. Calibração dos componentes

16. → Após colocar o código e colar as informações indicadas no passo número 10, aperte na seta horizontal do canto superior esquerdo para rodar o código.

→ Ao aparecer sinal de conexão segure o botão "***boot***" direto na placa **ESP32.**



→ Ao concluir o upload, abra o **serial** clicando no ícone de lupa localizado no canto superior direito.

→ Selecione no canto superior direito da área do serial "**115200 baud**", assim definindo a taxa da placa.

→ Aperte o botão "***EN***" direto na placa **ESP32.**

17. → Descobrir o valor de R0 (resistência em ar limpo), este é o coração da calibração.

Código de CALIBRAÇÃO

Use quando:

- É a **primeira vez** configurando o sensor
- Você percebe que o comportamento mudou muito (ex: a faixa de ar limpo que era 700-900 virou 1200-1430, como aconteceu com você)
- Trocou o sensor MQ-2 por um físico diferente
- Percebeu que o sistema está dando falso positivo/negativo com frequência, e suspeita que o R0 está desatualizado

→ O código abaixo lê o sensor em **ar limpo** (sem gás) e calcula o R0 médio:

```
#define PINO_MQ2 34
```

```
#define RL_VALUE 10.0
#define RO_CLEAN_AIR_FACTOR 9.83

float calcularRS(int leituraADC) {

  float tensao = leituraADC * (3.3 / 4095.0);
  if (tensao == 0) tensao = 0.01;
  float rs = ((3.3 * RL_VALUE) / tensao) - RL_VALUE;
  return rs;
}

void setup() {
  Serial.begin(115200);
  Serial.println("Aquecendo o sensor... aguarde");
  delay(20000);

    Serial.println("Calculando R0, mantenha o sensor em ar limpo...");

  float somaRS = 0;
  int numLeituras = 50;

  for (int i = 0; i < numLeituras; i++) {
    int leitura = analogRead(PINO_MQ2);
    float rs = calcularRS(leitura);
    somaRS += rs;
    delay(200);
  }

  float rsMedio = somaRS / numLeituras;
  float r0 = rsMedio / RO_CLEAN_AIR_FACTOR;

  Serial.println("====================================");
  Serial.print("RS medio em ar limpo: ");
  Serial.println(rsMedio);
  Serial.print("R0 calculado: ");
  Serial.println(r0);
  Serial.println("====================================");
    Serial.println("Anote esse valor de R0 e use no codigo principal!");
}
```

```
void loop() {
  // nao precisa fazer nada aqui, o resultado ja foi impresso no
setup
}
```

→ Ele vai imprimir no serial da seguinte maneira:

```
====================================
RS medio em ar limpo: 30.33
R0 calculado: 3.09
====================================
```

(Os resultados mostrados são apenas exemplos!)

→ Após calcular o valor de R0 com o código de calibração, use o código de diagnóstico para calcular o LIMIAR_GAS substituindo o valor de R0 no código.

Código de DIAGNÓSTICO

Use quando:

- Você já tem um R0 definido, e quer **ver os valores em tempo real** pra decidir o `LIMIAR_GAS`
- Está testando se o sensor está reagindo corretamente ao aproximar gás
- Quer confirmar se a fiação/pino está correto (ex: quando descobrimos que o fio estava no D32 mas o código lia o pino 34)
- Está debugando algum comportamento estranho (tipo "gás detectado infinito")

→ O código de diagnóstico:

```
#define MQ2_AOUT_PIN 34
#define RL_VALUE 10.0
float R0 = 2.33; // <-- troque pelo valor mais recente da sua
calibração
```

```
int lerMedia() {
  long soma = 0;
  int numLeituras = 20;
  for (int i = 0; i < numLeituras; i++) {
    soma += analogRead(MQ2_AOUT_PIN);
    delay(10);
  }
  return soma / numLeituras;
}

void setup() {
  Serial.begin(115200);
  delay(2000);
   Serial.println("Aquecendo... aguarde 60s antes de aproximar o
gas");
}

void loop() {
  int leituraADC = lerMedia();
  float tensao = leituraADC * (3.3 / 4095.0);
  if (tensao == 0) tensao = 0.01;
  float rs = ((3.3 * RL_VALUE) / tensao) - RL_VALUE;
  float razao = rs / R0;

  Serial.print("ADC medio: ");
  Serial.print(leituraADC);
  Serial.print(" | Tensao: ");
  Serial.print(tensao, 3);
  Serial.print("V | RS: ");
  Serial.print(rs, 2);
  Serial.print(" | Razao RS/R0: ");
  Serial.println(razao, 3);

  delay(500);
}
```

→ Ele vai imprimir no serial da seguinte maneira:

```
ADC medio: 823 | Tensao: 0.663V | RS: 15.32 | Razao RS/R0: 6.575
ADC medio: 831 | Tensao: 0.669V | RS: 15.11 | Razao RS/R0: 6.485
ADC medio: 819 | Tensao: 0.660V | RS: 15.42 | Razao RS/R0: 6.618
```

ADC médio	Leitura bruta do pino (0-4095), já suavizada pela média de 20 leituras
Tensão	A mesma leitura, convertida pra volts (0-3.3V)
RS	Resistência calculada do sensor naquele momento
Razão RS/R0	RS dividido pelo R0 fixo — é esse valor que você compara com o `LIMIAR_GAS`

→ Observe a Razão em ar limpo

Rode o Diagnóstico sem gás nenhum por perto, por uns 30 segundos, e veja os valores que aparecem, tipo:

```
Razao RS/R0: 6.575
Razao RS/R0: 6.485
Razao RS/R0: 6.618
Razao RS/R0: 6.520
```

Aqui, a faixa "normal" (sem gás) fica entre aproximadamente **6.4 e 6.6**.

→ Observe a Razão com gás perto

Aproxime o gás (com cuidado) e veja os valores mudarem, tipo:

```
Razao RS/R0: 2.310
Razao RS/R0: 2.180
Razao RS/R0: 2.402
```

Aqui, com gás, a faixa cai para **2.1 a 2.4**.

→ Escolha um número no meio das duas faixas

Nesse exemplo:

- Sem gás: ~6.4 a 6.6
- Com gás: ~2.1 a 2.4

Existe uma "zona de segurança" bem grande entre 2.4 e 6.4 — você pode escolher qualquer número nessa faixa, mas o ideal é ficar **mais ou menos no meio**, pra ter boa margem dos dois lados.

O caminho completo em resumo:

1. **Rode o código de Calibração** → ele te dá o **R0** (ex: `2.33`)
2. **Coloque esse R0** no código de Diagnóstico (na linha `float R0 = 2.33;`)
3. **Rode o código de Diagnóstico** com esse R0 atualizado:
- Observe a faixa de "Razão" em ar limpo (ex: 6 a 8)
- Aproxime o gás e observe a faixa de "Razão" com gás (ex: 2 a 3)
4. **Escolha um LIMIAR_GAS** no meio das duas faixas (ex: `5.0`)
5. **Coloque os dois valores no código principal** (o que tem o Blynk):

```
float R0 = 2.33;          // valor do passo 1 (Calibração)
float  LIMIAR_GAS  =  5.0;     //  valor  do  passo  4  (baseado  no
Diagnóstico)
```

Ok! Agora execute o código e faça os testes de detecção e notificação do blynk.

6. Guia de siglas

RS:

Vem de "Resistance Sensor" (ou às vezes referido como "Sensor Resistance") — é a resistência elétrica do sensor naquele exato momento, medida em tempo real.

Quanto mais gás no ar, menor fica a resistência RS (o material "conduz mais facilmente" na presença de gases combustíveis).

Quanto mais limpo o ar, maior fica RS.

R0:

Vem de "Resistance zero" ou "Resistance at 0 (referência)" — é a resistência do sensor medida em uma condição de referência: ar limpo. O "0" aqui não significa "resistência igual a zero", significa "ponto de referência/calibração inicial".

Por que dividir um pelo outro (RS/R0)?

Essa divisão serve para normalizar a leitura — ou seja, transformar o valor bruto (que varia de sensor pra sensor) em uma proporção universal, que representa "quanto a resistência atual está diferente da resistência em ar limpo".

Por que isso é melhor que usar só o RS puro?

Se você comparasse só o valor bruto de RS com um número fixo (tipo "se RS < 15, tem gás"), esse número não funcionaria em outro sensor MQ2 diferente do seu, porque cada um tem uma resistência de base diferente. Ao dividir por R0, você cria uma métrica que se ajusta automaticamente à característica única do seu sensor específico — é por isso que a calibração (achar o R0 certo) é tão importante antes de definir o limiar.

No seu projeto, esse gráfico de curvas Rs/Ro é inclusive o que o datasheet do MQ2 usa oficialmente pra relacionar a proporção com a concentração estimada de gás em partes por milhão (ppm) — mas pro seu caso (detector de vazamento simples), só usar a razão como indicador relativo de "mais perto ou mais longe do normal" já é suficiente.

Pra que serve calcular RS e R0?

Calculando essa proporção (RS/R0), você cria uma métrica que se auto-ajusta à condição atual do seu sensor específico — em vez de confiar em um número mágico que alguém achou em um tutorial na internet (que provavelmente não bate com o seu sensor).

Em resumo, o propósito prático é:
Confiabilidade: ter uma forma cientificamente validada (do próprio datasheet do fabricante) de detectar gás, em vez de "chutar" um limiar
Precisão: distinguir de forma clara entre "ruído normal do sensor" e "gás de verdade" (foi exatamente isso que resolveu seu problema de "gás detectado infinito")
Adaptação: se você trocar de sensor no futuro, ou o sensor "amadurecer" com o uso, basta recalibrar o R0 — o resto da lógica continua funcionando igual.